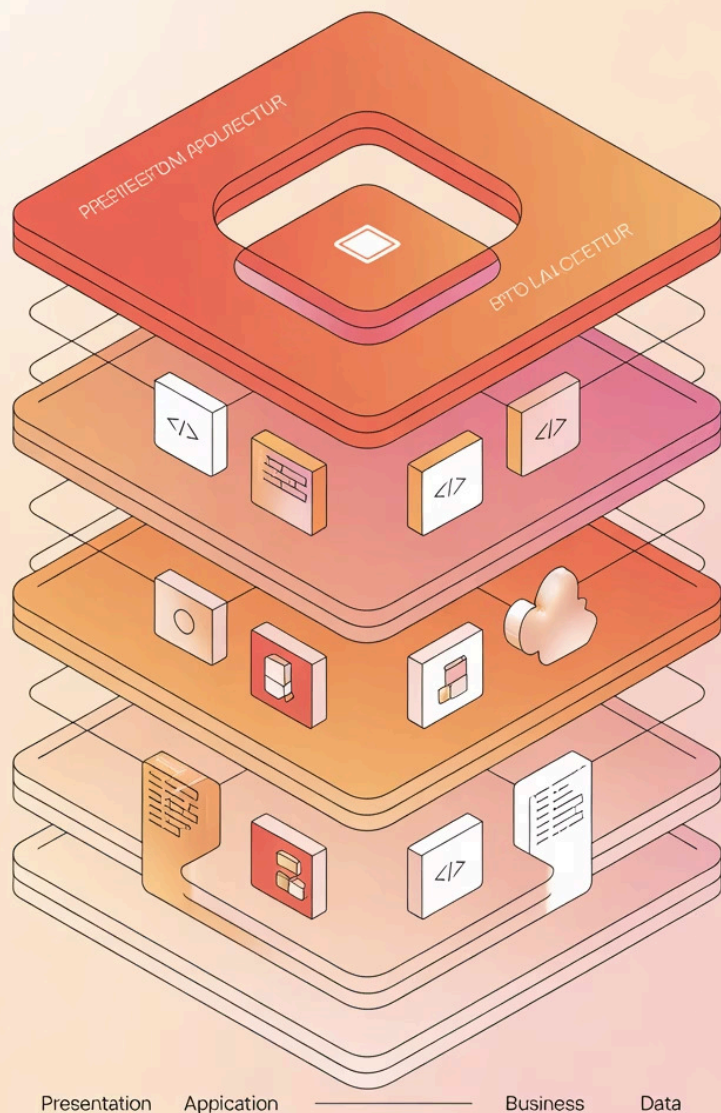
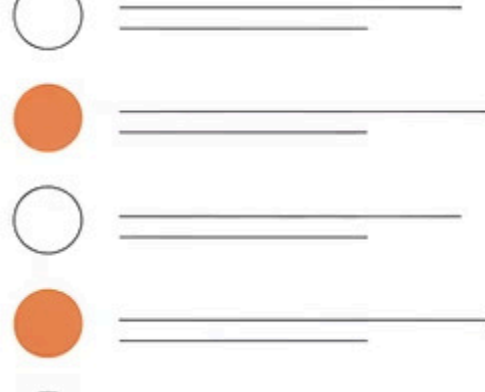




Capas en Spring Boot: Controller – Service – Repository

Video #5 - Serie: Java con Spring Boot de 0 a Pro

APIs REST con Spring Boot



¿Qué aprenderás hoy?

1 Separación de responsabilidades

Entenderás por qué es crucial organizar tu código en capas bien definidas para crear aplicaciones mantenibles y escalables.

2 Las tres capas fundamentales

Conocerás Controller, Service y Repository: qué hace cada una y cómo trabajan juntas en tu aplicación Spring Boot.

3 Implementación práctica

Crearás un flujo completo usando estas capas con datos en memoria, preparando el terreno para JPA en futuros videos.



¿Por qué organizar el código en capas?

Sin organización

Todo en el Controller:

- Validaciones mezcladas
- Lógica de negocio dispersa
- Acceso a datos directo
- Código difícil de mantener

Con capas organizadas

Responsabilidades claras:

- Cada capa tiene un propósito específico
- Código reutilizable y testeable
- Fácil mantenimiento y escalabilidad
- Sigue principios SOLID

La separación en capas es una práctica fundamental que te permitirá crear aplicaciones profesionales y mantenibles a largo plazo.

Controller: La puerta de entrada

Responsabilidad principal

Manejar las peticiones HTTP (GET, POST, PUT, DELETE) y devolver las respuestas apropiadas al cliente.

Lo que NO debe hacer

No debe contener lógica de negocio ni acceder directamente a los datos. Su único trabajo es coordinar.

Ejemplo típico

Recibe un POST para crear usuario, valida el formato, llama al Service y retorna la respuesta HTTP.

```
@RestController
@RequestMapping("/api/usuarios")
public class UsuarioController {

    @PostMapping
    public ResponseEntity crear(@RequestBody Usuario usuario) {
        Usuario creado = usuarioService.crear(usuario);
        return ResponseEntity.ok(creado);
    }
}
```



Service: El cerebro de tu aplicación

Lógica de negocio

Aquí van las reglas de tu aplicación: validaciones complejas, cálculos, transformaciones de datos y toda la inteligencia del sistema.

Orquestación

Coordina las operaciones entre diferentes repositorios, servicios externos, y aplica las reglas de negocio específicas.

Reutilización

Un mismo service puede ser usado por diferentes controllers, evitando duplicación de código y manteniendo consistencia.

```
@Service
public class UsuarioService {

    public Usuario crear(Usuario usuario) {
        // Validar datos de negocio
        if (usuario.getEdad() < 18) {
            throw new IllegalArgumentException("Usuario debe ser mayor de edad");
        }
        // Llamar al repository
        return usuarioRepository.guardar(usuario);
    }
}
```



Repository: Guardián de tus datos

Acceso a datos

Única responsabilidad: manejar la persistencia y recuperación de datos, ya sea en memoria, base de datos o servicios externos.

Abstracción

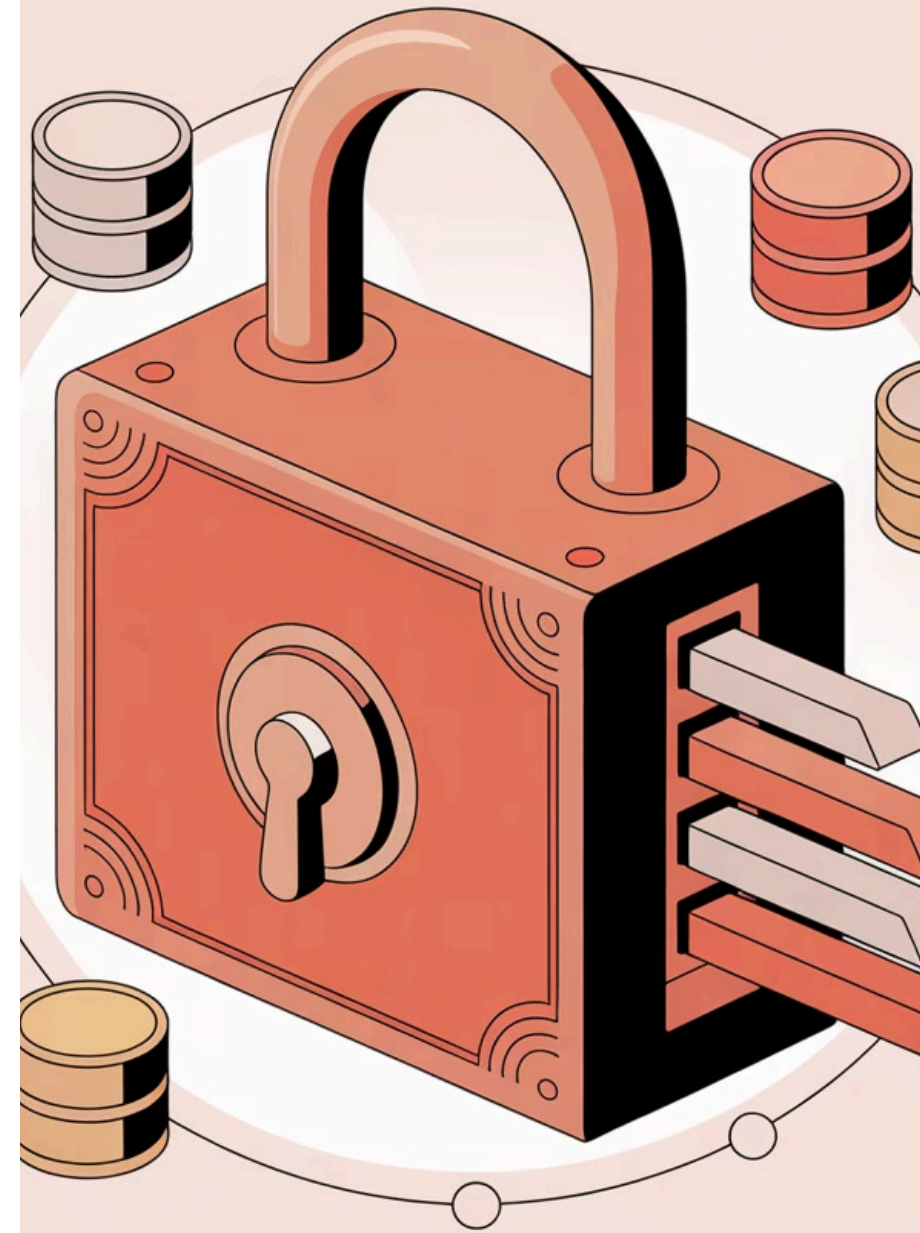
Ocultar los detalles de cómo se almacenan los datos. El Service no necesita saber si usas MySQL, MongoDB o memoria.

Por ahora: en memoria

Empezaremos con una implementación simple usando listas en memoria, perfecto para aprender sin complejidad extra.

@Repository

```
public class UsuarioRepository {  
    private List usuarios = new ArrayList<>();  
    private Long idCounter = 1L;  
  
    public Usuario guardar(Usuario usuario) {  
        usuario.setId(idCounter++);  
        usuarios.add(usuario);  
        return usuario;  
    }  
}
```



Flujo completo de la arquitectura



Este patrón garantiza una responsabilidad clara por capa, optimizando el mantenimiento y las pruebas de la aplicación.

Checklist para la demostración

01

Crear las clases

UsuarioController, UsuarioService y UsuarioRepository con sus respectivas anotaciones (@RestController, @Service, @Repository).

02

Implementar el flujo

Controller llama al Service, Service aplica validaciones y llama al Repository, Repository maneja la lista en memoria.

03

Configurar endpoints

POST para crear usuario, GET para listar usuarios, GET por ID para buscar usuario específico.

04

Probar con Postman

Verificar que todos los endpoints funcionan correctamente y que los datos se persisten en memoria durante la sesión.

 **Herramientas necesarias:** JDK 17+, Spring Boot 3.x, Maven, tu IDE favorito y Postman para las pruebas.

Spring Boot Setup



Configure dependencies



Apricot Pproject



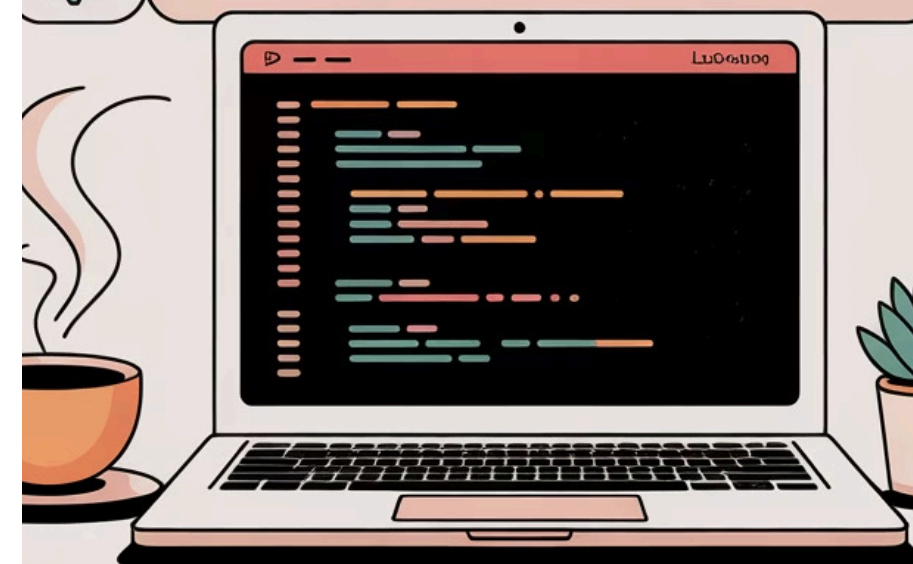
Apricot



Initialize Project



Ppdedropiott





Resumen y próximos pasos

Lo que aprendiste hoy

- **Separación de responsabilidades** en capas claras
- **Controller:** maneja HTTP requests/responses
- **Service:** contiene la lógica de negocio
- **Repository:** acceso y persistencia de datos

Próximos videos

- Integración con **JPA** y base de datos real
- Manejo de **excepciones** personalizadas
- **Validaciones** avanzadas con Bean Validation

Ya tienes las bases sólidas de una arquitectura profesional. En el siguiente video veremos cómo conectar tu Repository con una base de datos real usando JPA y Spring Data.

📝 **Practica:** Intenta crear un recurso diferente (como "Producto") aplicando la misma arquitectura de capas. ¡La práctica hace al maestro!